UNIVERSITY OF LONDON
INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects

CM3070 PROJECT
PRELIMINARY PROJECT REPORT

Alfred: Your Crypto AI Companion

Author: Koh Jun Hao
Student Number : 10230839
Date of Submission: 18th December 2025
Supervisor: Mr. Chew Jee Loong

Table of Contents

Chapter 1: Introduction *4*

1.1 Project Context and Motivation **4**

1.2 Project Aims and Research Questions **4**

1.3 Objectives and Deliverables **5**

Chapter 2: Literature Review *5*

2.1 Vulnerability to Misinformation **6**

2.2 Market Data Fragmentation **6**

2.3 Knowledge Cut-off and Hallucination **6**

2.4 High Barriers to Entry **7**

2.5 Analysis of Existing Solutions **7**

2.5.1 Social Sentiment and Narrative Agents 7

2.5.2 Autonomous Execution Agents 8

2.5.3 Natural Language Interfaces 8

2.5.4 The Black Box Issue 9

2.6 Agentic RAG and Chain of Thought **9**

2.6.1 Agentic RAG 9

2.6.2 Chain of Thought (CoT) Prompting 9

Chapter 3: Project Design *10*

3.1 Target Audience and User Classification **10**

3.1.1 User A: The Novice ("Simple Mode") 10

3.1.2 User B: The Expert ("Pro Mode") 10

3.2 System Architecture **10**

3.2.1 The Interface and Control Layer 11

3.2.2 The Intent Parser (The Router) 11

3.2.3 The Data Aggregation Layer 11

3.2.4 The Synthesis Layer (LLM Engine) 12

3.3 Data Flow and Logic **12**

3.3.1 Step 1: Input Reception and Validation 12

3.3.2 Step 2: Intent Classification 12

3.3.3 Step 3: Data Aggregation 12
3.3.4 Step 4: Context Injection 12

3.4 Justification of Technical Stack 13
3.4.1 Backend Framework: Flask (Python) 13
3.4.2 Inference Engine: Ollama (Llama 3) 13
3.4.3 APIs (CoinGecko & DefiLlama) 13
3.4.4 Frontend: Tailwind CSS 14

3.5 Project Planning and Methodology 14
3.5.1 Development Methodology 14
3.5.2 Gantt Chart 14

3.6 Evaluation Strategy 15
3.6.1 Real Time Data (RQ1) 15
3.6.2 Simple/Pro Mode Effectiveness (RQ2) 15
3.6.3 Latency Testing (RQ3) 16

Chapter 4: Feature Prototype 16

4.1 Prototype Description 16

4.2 Technical Implementation 17
4.2.1 Intent Parsing (Regex Logic) 17
4.2.2 Data Aggregation (Threading) 17
4.2.3 Inference Engine 17

4.3 Preliminary Findings 18
4.3.1 System Performance 19
4.3.2 Edges Cases 19

Chapter 5: Appendixes 19

Appendix A: Full System Prompts (LLM Instructions) 19

Appendix B: API Data Aggregation 21

Appendix C: Intent Parser 21

Chapter 6: References 22

Chapter 1: Introduction

1.1 Project Context and Motivation

The cryptocurrency market is heavily volatile and complex. This differs from traditional finance (TradFi) where it is highly regulated. In cryptocurrency, the ecosystem is fragmented. It splits across decentralized exchanges, blockchains, and social media. Research by Raheman et al. (2022) states that the cryptocurrency market is "overly influenced by sentiment" rather than fundamental metrics, making investors susceptible to misinformation and "pump and dump" schemes. This creates a risk for retail investors because they lack verifiable data in real time, which leads to financial loss.

New adopters also face a barrier to entry. A knowledge moat exists because of technical jargon. Lewis (2018) argues that these terms exclude beginners from the digital economy, like slippage, gas fees, and TVL or Total Value Locked. Users often ask Large Language Models or LLMs like ChatGPT for help, but these models have a knowledge cut off problem. OpenAI documentation (2023) confirms that pre-trained models cannot access market data in real time without tools. They cannot answer time sensitive queries effectively, like "What is the price of Bitcoin right now?" or "Is the trend bullish?".

Alfred: Your Crypto AI Companion, named after Batman's trusted butler, fixes three issues: misinformation, jargon, and stale data. It is an AI Chatbot designed for the cryptocurrency market, and uses inspiration from the template "4.2 Project Idea 2: Financial Advisor Bot" from the CM3020 Artificial Intelligence module. Alfred works by building a retrieval augmented chatbot that is hosted locally. Standard chatbots rely on training data, but Alfred is uses a Python based Flask backend to enforce deterministic routing logic before engaging the LLM. It takes live data from sources i.e. CoinGecko and DefiLlama and uses a local Large Language Model (Ollama/Llama 3) to turn this data into natural language. Alfred handles the different expertise levels of users with a toggle, Simple Mode and Pro Mode. Simple Mode uses analogies for beginners and Pro Mode provides raw data tables for experts.

1.2 Project Aims and Research Questions

The main goal of this project is to build an Intent Based Retrieval Augmented Generation system, or RAG. This system will help to reduce LLM hallucinations, and also lower the entry barrier for cryptocurrency investors by adapting its output to the technical skills of the user. To reach this goal, the project answers these Research Questions or RQs:

1: How can we design a local LLM to prioritize real time data? It must value API fetched data over its pretrained knowledge base to prevent financial hallucinations.

2: How much does the Simple/Pro toggle improve understanding? This feature changes system prompts between Novice and Expert modes. We basically evaluate two user groups if this makes blockchain data clearer for users with different technical knowledge.

3: Can a locally hosted reasoning engine like Ollama achieve acceptable latency? When combined with a Intent Parser, can it respond in under 10 seconds for real time financial queries?

1.3 Objectives and Deliverables

To achieve the primary aim of creating an accurate cryptocurrency assistant, I have broken down the project into four technical objectives and deliverables.

The first objective is to build a backend system. It must distinguish between general conversation and queries specific to data. Standard chatbots rely on probabilistic token generation for facts. Alfred instead uses a Intent Parser to identify specific entities like "Bitcoin" and intents like "Get Price" or "Get TVL". This engine queries external APIs to retrieve market data in real time.

The deliverable is a Python based Flask application. It implements a Regex based intent_parser.py and a data_fetcher.py module. This module aggregates metrics from CoinGecko and DefiLlama.

The second objective is to solve the jargon barrier. I will implement a system that alters the output style based on user preference. This involves engineering distinct System Prompts for the LLM. These prompts enforce specific tones like "Explain like I am 5" or "Institutional Analyst". The injected data is always the same for all prompts.

The deliverable is a fully functional user interface which features a Simple and Pro toggle. This interface connects to a local reasoning engine, llm_engine.py. This engine should be capable of producing a response that has context.

The third objective is to optimize the architecture. Alfred needs to ensure that the combined latency remains within acceptable limits for the user to effectively use it for cryptocurrency data. This includes the Intent Parser, API calls, and local LLM generation. For this, I have set the limit to be under 10 seconds for a conversational experience.

The deliverable is a set of latency tests. These tests compare response times between Simple and Pro modes and ensure that they are below the limit (or find a reason why it is unable to).

The final objective is to validate the effectiveness of the system. The Pro Mode must deliver accurate raw data (100% of the time) without hallucination. The data must match the API source. It also verifies that Simple Mode lowers the barrier to entry for novice users.

The deliverable is a comprehensive evaluation chapter. It details accuracy rates (checked against raw data output) or hallucination checks. It also includes user feedback from two distinct user groups, Beginner and Expert.

Chapter 2: Literature Review

The financial landscape of cryptocurrency operates on very different conditions compared to Traditional Finance (TradFi). This difference creates a unique set of information challenges that existing tools struggle to address as it is a very nascent industry.

2.1 Vulnerability to Misinformation

In TradFi, market data is centralized and regulated through verified terminals like Bloomberg or Reuters. However, the cryptocurrency ecosystem is permissionless. Information flows through unregulated social media channels like X and Telegram. Research by Kutle (2021) indicates that the crypto market is vulnerable to news misinformation. Asset prices are largely influenced by sentiment rather than fundamental metrics like revenue or utilization. This is different from the stock market where quarterly earnings reports determine valuations. Cryptocurrencies trade heavily on narrative momentum which increases the risk of market manipulation. Raheman et al. (2022) analyzed social media sentiment and found that misinformation circulates rapidly. This happens because retail investors lack technical expertise. Malicious actors exploit this asymmetry to orchestrate pump and dump schemes. In these schemes, fake news drives artificial volume before a sell off. This literature validates the core design of Alfred. Alfred does not rely on social sentiment analysis which amplifies noise, and instead it relies on verifiable API data from trusted aggregators to provide objective truth.

Recent research by Li et al. (2024) confirms that this structural deficit makes the market "overly influenced by sentiment" rather than fundamental metrics. Unlike the stock market, where quarterly earnings r determines valuations, cryptocurrency assets trade on "narrative momentum." Li et al.'s study of Bitcoin, Ethereum, and Dogecoin demonstrated that asset prices exhibit a correlation with social media activity rather than on chain metrics. This validates the "greater fool theory" in crypto markets, where value is driven by the collective belief of the crowd rather than raw data.

2.2 Market Data Fragmentation

During my research, I found another issue of current market data which is data fragmentation. For Tradfi, the New York Stock Exchange (NYSE) acts as a central clearinghouse. However, crypto trading happens across hundreds of venues at the same time. This includes Centralized Exchanges (CEXs) like Binance which hold off chain order books, Decentralized Exchanges (DEXs) like Uniswap, which executes orders on chain, and DeFi Protocols like AAVE which is a lending and borrowing platform.

For a cryptocurrency analyst, gaining a day-by-day bird's eye view of the market requires navigating multiple complex interfaces: a block explorer for on chain activity, a CEX for price action, and a dashboard like DefiLlama for Total Value Locked (TVL). This fragmentation creates a steep learning curve, because beginners don't even know where to start or how to read the data. Alfred solves this by using data_fetcher.py to unify these sources. It fetches price data from CoinGecko and TVL from DefiLlama and combines them into a single response for the user.

The 2026 Crypto Crime Report by TRM Labs also highlights that while illicit activity is a concern, the actual volume of legitimate dispersed activity is overwhelming. The report notes that liquidity is no longer sitting solely on Ethereum Mainnet; it has migrated to high-performance L2s. For example, the Base network alone processed over 4.4 billion transactions in 2025, capturing 62% of total Layer 2 revenue. For an investor, this means that the price is not a single number but a function of liquidity across Arbitrum, Optimism, Base, and Mainnet, each with potentially different slippage rates and fees.

2.3 Knowledge Cut-off and Hallucination

Large Language Models or LLMs lead in natural language processing. However, they are unsuitable for real time financial advice due to the knowledge cut off problem. They are trained on a static dataset with a fixed end date. OpenAI documentation (2023) confirms that their trained models cannot access real time data without external tools.

So, if a user asks a question requiring real time data i.e. "What is the price of Bitcoin now?", the LLM hallucinates and generates a price that looks realistic like "$42,000" but this price is incorrect. In a financial context, this is a critical failure. Alfred fixes this by using a Router Based Architecture. This is explained in later chapters. The LLM is never allowed to recall a price from training data. Instead, the llm_engine.py injects live API data directly into the prompt context. This forces Alfred to act as a summarizer rather than a generator.

Sert (2025) also categorized these errors into "Factuality Hallucinations," where models invent numbers that are statistically likely but factually wrong. For example, if asked for the price of Bitcoin, a model might output "$60,000" not because it knows the price, but because that number appears frequently in its training data distribution.

2.4 High Barriers to Entry

Besides data, accessibility for the layman user is also a problem. Lewis (2018) argues in The Basics of Bitcoins and Blockchains that the industry relies too much on technical jargon. This includes terms like slippage, gwei, and impermanent loss which creates a knowledge moat that excludes users without technical skills. If Alfred just responds with terms like FDV or Fully Diluted Valuation, it does not help a novice.

Existing solutions like AIXBT and Elfa fail to fix this. They provide complicated data or explanations that hallucinate. This highlights the need for the Simple/Pro Mode in Alfred, which uses the same verified data point to generate two outputs. It makes a simple analogy for the novice like comparing Market Cap to the total value of a company. For the expert, it will provide raw yet actionable data.

2.5 Analysis of Existing Solutions

I will evaluate the current state of cryptocurrency AI assistants to explain why Alfred is needed. The market splits into three categories. These are social sentiment bots, autonomous execution agents, and raw data engines, with each category having their own limitations.

2.5.1 Social Sentiment and Narrative Agents

The most common crypto AI tools leverage Natural Language Processing (NLP). They scrape and analyze social media feeds to gauge market sentiment. AIXBT is an example which functions as an autonomous agent. It monitors Key Opinion Leaders (KOLs) to detect narrative shifts. For Elfa.AI, it is a social listening tool which aggregates signals from private Telegram groups.

Their approaches detect hype but it has reliability issues. Research by Liu and Tsyvinski (2021) indicates that bots can manipulate social feeds. This means these tools often amplify noise rather than signal. Also, access to these tools requires payment. Both AIXBT and Elfa.AI operate as paywalled communities. They require users to hold specific tokens like the AIXBT token to access the pro terminals. This financial barrier excludes the demographic that needs protection the most, the novice investor. Alfred will address this by rejecting social sentiment as a primary source of data and adopt a model that is focused on objective data.

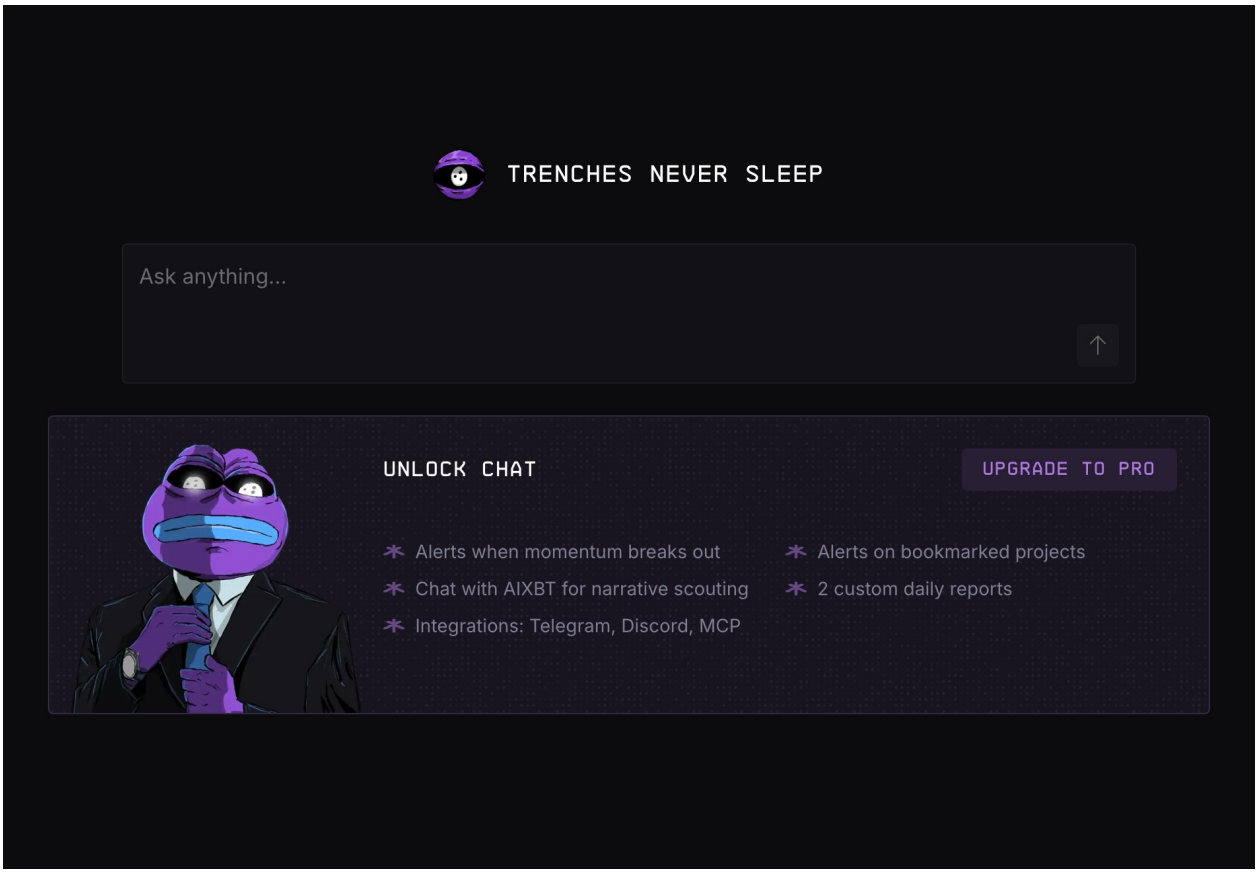

Figure 1: AIXBT's frontend (paywalled)

2.5.2 Autonomous Execution Agents

The second category involves agents built for transaction execution, i.e. HeyAnon.AI. It uses LLMs to interpret user intents and execute actions on chain. This covers actions like bridging assets or swapping tokens via protocols like LayerZero.

This approach is not ideal as it puts action before understanding. For a new user, they won't understand why and how the action was done. If the model misunderstands a prompt (i.e. confusing a limit order with a market order), it can cause financial loss. Alfred takes another approach and chooses education over execution. The Simple Mode toggle ensures concepts are explained via analogy (bridging is like moving money between two banks) before a user decides on their actions themselves.

2.5.3 Natural Language Interfaces

The third category includes tools like LlamaAI by Defillama (the same one Alfred is using for API calling). LLamaAI acts as a natural language interface for the DefiLlama database and is very accurate for retrieving metrics like Total Value Locked (TVL).

However, this data lack synthesis. It returns tables of numbers without context. i.e. a table showing "Aave TVL: $5B" which means nothing to a novice who does not understand what

TVL implies for protocol solvency. Also, LLamaAI is restricted to their own databases. LlamaAI is good at DeFi statistics but cannot give market context like price action from centralized exchanges like CoinGecko. But, Alfred serves as an aggregation layer and pulls data from multiple sources like CoinGecko for price and DefiLlama for TVL. It generates this into an output that both novices and experts can understand.

2.5.4 The Black Box Issue

An issue arising across all three categories (sentiment bots, execution agents, and data interfaces) is the closed source nature. They operate as black boxes and the user cannot verify the data source, the algorithm, or the system prompt governing the output.

This opacity is a risk in financial settings. A user cannot confirm if a recommendation is based on data or if the model is hallucinating. However, as Alfred is open source and auditable, with the System Prompts clearly defined in llm.py, this allows users to verify how the AI is instructed to behave.

2.6 Agentic RAG and Chain of Thought

Alfred is not just a chatbot connected to an API. It is an implementation of two AI frameworks: Agentic Retrieval Augmented Generation (Agentic RAG) and Chain of Thought (CoT) Prompting. These frameworks address the latency and hallucination noted in Sections 2.1 and 2.2.

2.6.1 Agentic RAG

Standard RAG involves embedding static text documents like PDFs into a vector database. When a user queries the system, a semantic search retrieves text chunks to augment the knowledge of the LLM.

However, this approach does not suit the cryptocurrency industry well. Financial data is volatile, and a database with yesterday's Bitcoin price is just misinformation. Therefore, Alfred implements a variation known as Router based Agentic RAG.

As an autonomous agent, Alfred will decide which tool to use before generating an answer. As implemented in intent_parser.py, the architecture replaces the probabilistic retrieval of a vector DB with a Intent Router.

2.6.2 Chain of Thought (CoT) Prompting

To mitigate the hallucination issue described in Section 2.2.1, Alfred leverages Chain of Thought (CoT) prompting. CoT is a technique where the Large Language Model generates intermediate reasoning steps before producing the final answer. Research suggests that breaking complex tasks into intermediate steps improves the logical consistency of LLM outputs.

Alfred uses this theory within its system code. The llm_engine.py module defines a strict reasoning within the system prompt. Before Alfred answers a user, it must follow the logic

follow of examining the provided data, identifying key metrics, and then consider what to reply for the novice.

By forcing the model to consider the data before giving an output, the framework constrains the model. It transforms the generation task from a creative task (which leads to hallucination) into an analytical task (which leads to accuracy). This implementation distinguishes Alfred from generic chatbots which do not have guardrails.

Chapter 3: Project Design

3.1 Target Audience and User Classification

To address the knowledge moat issue identified in the Literature Review, Alfred is designed to serve two distinct user demographics. These groups have different requirements for using Alfred. To solve this, the system implements a Simple/Pro Mode that alters the backend system prompt based on the user selection.

3.1.1 User A: The Novice ("Simple Mode")

Simple Mode caters to users with limited technical knowledge. They are often overwhelmed by the complexity of crypto jargon and now sure how to use them effectively. They do not need raw data tables but rather context. When Simple Mode is active, the system injects a specific instruction set into the LLM, which is shared in Chapter 5. The output is limited to 2 or 3 sentences to prevent confusion.

3.1.2 User B: The Expert ("Pro Mode")

Pro Mode caters to users like high frequency traders or institutional analysts. They are familiar with market mechanics so chatty AI explanations are not useful for them. They require access to metrics such as Volume, Market Cap and Fully Diluted Valuation (FDV) to make execution decisions. In Pro Mode, the system prompt is different from Simple Mode and the output is data focused.

3.2 System Architecture

The technical architecture of Alfred is designed as an Agentic RAG System. Unlike traditional chatbot architectures that rely on a single LLM to handle knowledge retrieval and generation, Alfred separates these functions into different layers.

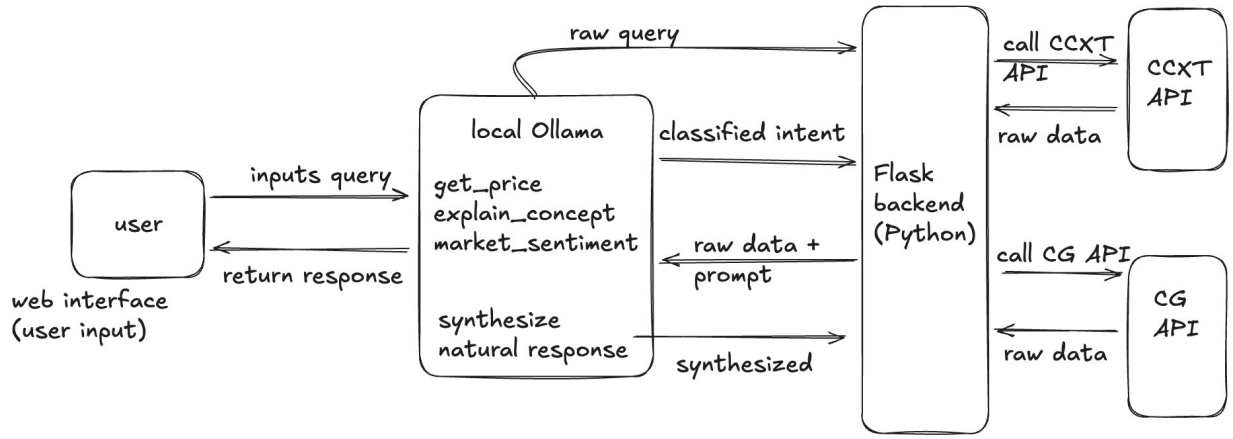

Figure 2: Alfred's High Level Architecture

The system is organized into four primary layers.

3.2.1 The Interface and Control Layer

The entry point of the system is a web interface built with HTML5 and TailwindCSS. It communicates with a Flask backend. The user submits a query via the frontend. This is captured by the /api/chat endpoint in routes.py. The Flask server acts as the central controller. Upon receiving a POST request, it extracts the message and the active mode (Simple or Pro). It then passes the raw input to the Reasoning Layer first. This implementation filters out invalid requests before they reach the inference engine.

3.2.2 The Intent Parser (The Router)

The Intent Parser module (intent_parser.py) determines the accuracy of Alfred. It classifies user input into three distinct intent types: Price Check, DeFi Stats, and General Chat. This classification step is important because it dictates which external tool Alfred will employ. For example, it will prevent the LLM from attempting to answer a math question with text generation.

3.2.3 The Data Aggregation Layer

Once the intent and entity (i.e. "Ethereum") are identified, the system activates the data_fetcher.py module. This layer handles the Retrieval in RAG. Alfred implements a get_combined_metrics() function that queries multiple APIs in parallel using a ThreadPoolExecutor. The CoinGecko API is used to fetch real time price, market cap, and 24 hour volume. The DefiLlama API is used to fetch metrics like Total Value Locked (TVL) data. The result is a unified JSON object (i.e. {'price': 3000, 'tvl': 50000000}) that serves as the truth for the next layer.

3.2.4 The Synthesis Layer (LLM Engine)

The final layer is the LLM engine (llm_engine.py). Alfred hosts a local instance of Ollama (Llama 3). It constructs a prompt that wraps the JSON data from the Aggregation Layer with specific instructions based on the selected user mode. The LLM does not recall facts, it only processes the injected JSON and translates it into natural language (either a simple analogy or a professional table). This ensures the output is always anchored to the latest API timestamp which solves the knowledge cutoff issue.

3.3 Data Flow and Logic

This details the complete cycle of a user query. The process follows a sequence defined in the chat() function within routes.py. Also, errors at any stage like an API timeout) will be caught and handled without crashing the application.

3.3.1 Step 1: Input Reception and Validation

When the user submits a text string via the frontend ("What is the price of Ethereum?"), the Flask server receives a POST request containing the message and the current mode. The input is stripped of whitespace and then validated. Empty requests will be rejected with a 400 Bad Request error to prevent processing.

3.3.2 Step 2: Intent Classification

Before any data is fetched, Alfred must understand what is being asked. The raw message is passed to the determine_intent() function in intent_parser.py. The parser applies a series of Regular Expressions to the input. It first checks for DeFi specific keywords. If none are found, it checks for Price keywords. Simultaneously, the parser extracts the target asset. For the query "price of Solana," the parser filters out stop words ("what", "is", "the", “of”) to isolate the Solana entity.

3.3.3 Step 3: Data Aggregation

Upon receiving a valid entity, the controller calls get_combined_metrics(entity) from data_fetcher.py. To minimize latency, the system uses a ThreadPoolExecutor to query CoinGecko (for market data) and DefiLlama (for TVL) simultaneously. If a direct ID match fails (i.e. the user types "Sol" instead of "Solana"), the search_coingecko() function is triggered to find the closest matching asset ID. A consolidated JSON object is returned containing all available metrics (Price: $150, TVL: $4B, Source: CoinGecko+DefiLlama).

3.3.4 Step 4: Context Injection

In the final stage, the raw data meets the router. Alfred selects the appropriate system prompt based on the user mode (Simple vs Pro). The fetched JSON data is injected into a structured message format:

User Question: What is the price of Solana?

Real Time Data (Source: CoinGecko):

{ "price": 150.20, "change_24h": 5.4 }

This text block is passed to query_ollama() in llm_engine.py. Because of the Chain of Thought instructions, it analyzes the JSON and generates a natural language response (i.e. "Solana is currently trading at $150.20...") without hallucinating the numbers. The final text is then sent back to the frontend as a JSON response. The Javascript should render it as a chat bubble.

3.4 Justification of Technical Stack

The selection of technologies for Alfred was driven by the non functional requirements defined in Chapter 1: low latency and transparency.

3.4.1 Backend Framework: Flask (Python)

The core controller is built using Flask, a lightweight Python web framework. While Django was considered, it provides a monolithic structure, and has middleware that adds latency. Alfred requires a lean, high performance layer to route requests between the frontend, the API fetcher, and the LLM. The minimalist design of Flask allows for the implementation of intent_parser.py without the overhead of a full stack framework.

Python was selected as the runtime environment due to its overall dominance as a programming language. This ensures compatibility with data libraries and the requests module used for my API aggregation.

3.4.2 Inference Engine: Ollama (Llama 3)

A critical decision was to host the Large Language Model locally using Ollama, specifically the Llama 3 model. As mentioned in Section 2.3, current solutions often operate as black boxes. By running Llama 3 locally, Alfred ensures that the process is fully transparent and that no user data is sent to third party providers like OpenAI.

Local inference also eliminates token API costs. This makes the system sustainable for my project and scalable for Pro Mode users who may generate thousands of queries.

Ollama also allows for control over the temperature parameter (I set this to 0.7 in llm.py). This strikes a balance between analogies for Simple Mode and rigid data for Pro Mode.

3.4.3 APIs (CoinGecko & DefiLlama)

To achieve the aggregation of data, Alfred utilizes standard APIs rather than WebSocket connections or web scraping.

CoinGecko API was chosen for its coverage of off chain market data, including Market Cap and Fully Diluted Valuation (FDV), which are essential for the Pro mode. DefiLlama API was

selected as the industry standard for decentralized metrics (TVL). Personally, I have used their APIs extensively before and they are robust.

As seen in data_fetcher.py, the system uses a ThreadPoolExecutor to query these endpoints concurrently. This ensures that the response time is limited only by the slowest API. This prevents a bottleneck where the user waits for one data source before the other is requested.

3.4.4 Frontend: Tailwind CSS

The user interface is built using Tailwind CSS. This framework was chosen to prototype a modern aesthetic since it is for a nascent market like cryptocurrency. And importantly, it handles the dynamic changes of the Modes (switching visually from Simple to Pro) without writing complex custom CSS classes.

3.5 Project Planning and Methodology

3.5.1 Development Methodology

For Alfred, I will use an Agile (Iterative and Incremental) development methodology. Given the complexity of integrating multiple APIs with a probabilistic LLM, I am not using a Waterfall approach. Agile allows for continuous testing of the Intent Router logic. If I discover new edge cases from users (like users asking for "Eth" instead of "Ethereum"), the Regex patterns can be updated in the next sprint without redoing the architecture.

The development lifecycle is divided into two week sprints. This ensures that a shippable feature (Price Fetching or Simple/Pro Mode) is completed and tested at the end of each cycle.

3.5.2 Gantt Chart

The Gantt Chart is structured around the key submission deadlines defined in the CM3070 guidelines.

Phase 1: Inception and Architecture (Weeks 1-10)

Focus: Definition of scope, literature review, and setting up the Flask and Ollama environment.
Milestone: Submission of Preliminary Project Report (PPR).
Status: Completed. The core Router architecture (routes.py, intent_parser.py) is implemented and serving basic requests.

Phase 2: Core Feature Implementation (Weeks 11-16)

Focus: Refining the Simple/Pro Mode and expanding API coverage.
Tasks: Sprint 3 (Weeks 11-12): Optimize llm.py prompts. Implement Simple Mode analogy generation to ensure it simplifies complex metrics.
Sprint 4 (Weeks 13-14): Build an impressive Pro Mode. Enhance data_fetcher.py to pull off chain metrics (FDV, Market Cap) from CoinGecko and on chain metrics (TVL) from DefiLlama simultaneously.

Sprint 5 (Weeks 15-16): Frontend Polish. Finalize the Tailwind CSS Glassmorphism UI and ensure the chat interface handles asynchronous loading states.

Phase 3: Testing and Evaluation (Weeks 17-20)

Focus: Verification of Research Questions (Accuracy, Latency).
Tasks: Run the accuracy tests (comparing API JSON vs LLM Text).
Conduct User Acceptance Testing (UAT) with the two user groups (Novice vs Expert).
Optimize latency if average response time exceeds 10 seconds (potential solution: caching API responses).

Phase 4: Final Reporting (Weeks 21-24)

Focus: Academic writing and code documentation.
Milestone: Submission of Final Project Report and Code.

3.6 Evaluation Strategy

To answer the Research Questions (RQs) defined in Chapter 1, I will undergo a three part evaluation process. This evaluation will move beyond simple debugging and provide quantitative proof that Alfred does not have hallucination and latency problems identified in the Literature Review.

3.6.1 Real Time Data (RQ1)

The main risk identified in the Literature Review is the tendency of LLMs to hallucinate numerical data (RQ1). To verify that Alfred mitigates this, I will conduct a Ground Truth Comparison test. A dataset of 10 specific queries (i.e. "Price of Solana," "TVL of Curve") will be executed against the system. The natural language output of Alfred will be checked against the raw JSON timestamp and value returned by the internal API log.

Alfred must achieve a 0% hallucination rate for numerical values. If the API returns $150.20, Alfred must generate "$150.20" (or a rounded equivalent in Simple Mode) without deviation. Any difference in data will be marked as a critical failure.

3.6.2 Simple/Pro Mode Effectiveness (RQ2)

To evaluate the educational value of the Simple/Pro Mode (RQ2), user testing will be conducted with two distinct groups.

Group A (Novices) will be 5 participants with no prior crypto experience. They will use Simple Mode to ask about complex topics for them (i.e. "What is TVL?"). They will be surveyed on clarity and understanding.

Group B (Experts) will be participants with active experience in the cryptocurrency industry. They will use Pro Mode to request market data. They will be surveyed on data density and efficiency.

This tests the specific prompt engineering found in llm.py. It needs to confirm that the instructions to "Use Analogies" effectively lower the barrier to entry for novices. It also needs to confirm that the instructions to "Be robotic and precise" satisfy the requirements for experts.

3.6.3 Latency Testing (RQ3)

To determine the feasibility of a local reasoning engine (RQ3), I will time the latency of the system to ensure the Python backend does not introduce delays.

I will use the Python time module to log the execution duration of each layer (intent parsing, data retrieval, and inference) for 10 requests. The total time for all these layers must be under 10 seconds to justify the use of Ollama.

Chapter 4: Feature Prototype

4.1 Prototype Description

For this chapter, I developed a prototype to demonstrate the feasibility of Alfred. Instead of developing all potential and more complicated features, this prototype focuses on two features: the real time analysis of data and the switching of Simple Mode and Pro Mode.

The main feature of the prototype is the ability to distinguish between financial queries using the Router architecture. The system accepts natural language queries (i.e. "check btc price" or "how much is solana worth") and extract the asset ticker. It then retrieves a market snapshot, including current price, 24 hour change, and trading volume. Beyond pricing, Alfred also supports queries regarding DeFi protocols ("TVL of Curve"). The system identifies this and routes the request to the DefiLlama API instead of the Coingecko API.

This feature will demonstrate that the Intent Parser filters user noise and triggers the correct Python functions without reliance on the LLM for logic.

The second feature is the implementation of the 2 modes. This answers Research Question 2 (RQ2) regarding accessibility. The frontend implements a state aware toggle switch allowing users to select between SIMPLE and PRO modes. This selection is transmitted as a JSON payload ({mode: 'PRO'}) with every API request. Upon receiving the change in mode, the backend swaps the system prompt used by the LLM.

Simple Mode activates instructions to "use analogies" and "round numbers." This ensures the output is educational. Pro Mode activates instructions to "be robotic" and output a "Markdown Table." This ensures the output is analytical.

This feature will demonstrate that a single underlying data source (the JSON from CoinGecko) can be changed into two different user outputs through prompt engineering.

4.2 Technical Implementation

The prototype functionality is achieved through three technical implementations: anintent parser, a multi-threaded data aggregator, and an inference engine. The following sections detail the code logic used to achieve these features.

4.2.1 Intent Parsing (Regex Logic)

To implement the Router architecture, intent_parser.py utilizes Regular Expressions (Regex) to classify user input before it reaches the inference layer.

In the determine_intent function, the system first cleans the input by removing punctuation and converting it to lowercase. For DeFi, the system scans for high specificity keywords using the pattern r'(tvl|locked|yield|apy|defi)'. If no DeFi intent is found, it falls back to a broader pattern r'(price|value|cost|worth|buying|selling|usd)'.

To identify which asset to query, the code tokenizes the string and filters out a predefined list of stop words (i.e. "what", "is", "the"). The remaining tokens are then analyzed and Alfred selects the most likely candidate (for the query "check tvl of curve," the parser filters "check," "tvl," and "of," leaving "curve" as the entity).

This implementation allows Alfred to process intents in near constant time (O(n) relative to string length). This minimizes latency before the network request.

4.2.2 Data Aggregation (Threading)

An important requirement for the Pro Mode is the display of a comprehensive market view. Fetching this data sequentially (Price to Volume to TVL) would introduce unacceptable latency.

So to solve this, the data_fetcher.py module implements the get_combined_metrics function using the concurrent.futures library in Python. It submits two simultaneous tasks, get_rich_market_data(entity) (to use CoinGecko API) and get_defi_tvl(entity) (to use DefiLlama API). The results are collected via future.result() and merged into a single dictionary. This ensures that the total fetch time is limited to the duration of the slowest single API call.

4.2.3 Inference Engine

The Simple/Pro Mode is implemented in the llm_engine.py module. The code defines dictionary system prompts containing two distinct instructions. Simple instructs the model to "Use plain English without jargon" and "Write 2-3 clear sentences." Pro instructs the model to "Be precise and robotic" and "Output the data in a Markdown Table."

When query_ollama is called, it constructs a composite prompt:

```
full_prompt = (

    f"System: {system_prompt}\n\n"
```

```
    f"User Data Context:\n{user_message}\n\n"

    f"Alfred Response:"

)
```

This structure sandwiches the raw API data between the personality instructions and the generation trigger. This forces the local Llama3 model to process the provided JSON according to the active mode.

4.3 Preliminary Findings

I conducted initial testing of Alfred to verify it can generate contextual responses from the data source given. The results show that Simple/Pro Mode is working properly and the data retrieved is accurate.

To test the effectiveness of the prompts written in llm.py, the same query "What is the price of Bitcoin?" was submitted to Alfred in both Simple and Pro modes. The query is identical, yet the difference in response is proof the prompt injection works.

In Simple Mode, this was what occurred:

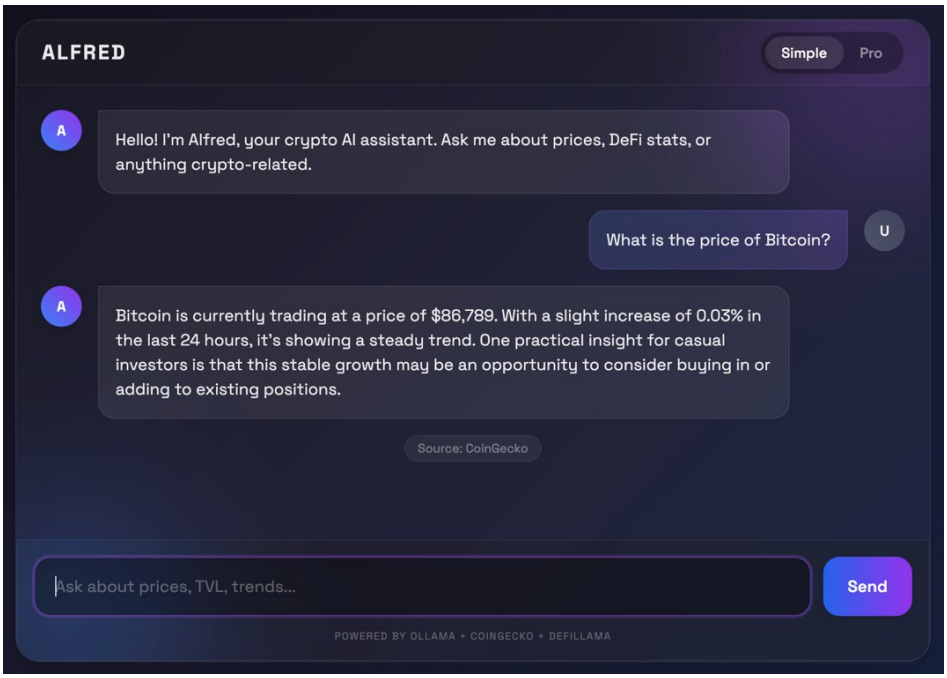

Figure 3: Simple Mode test in Alfred

Observation: The model correctly rounded the decimals (ignoring the exact cents) and suggests what the price might mean.

In Pro mode, this was what occurred:

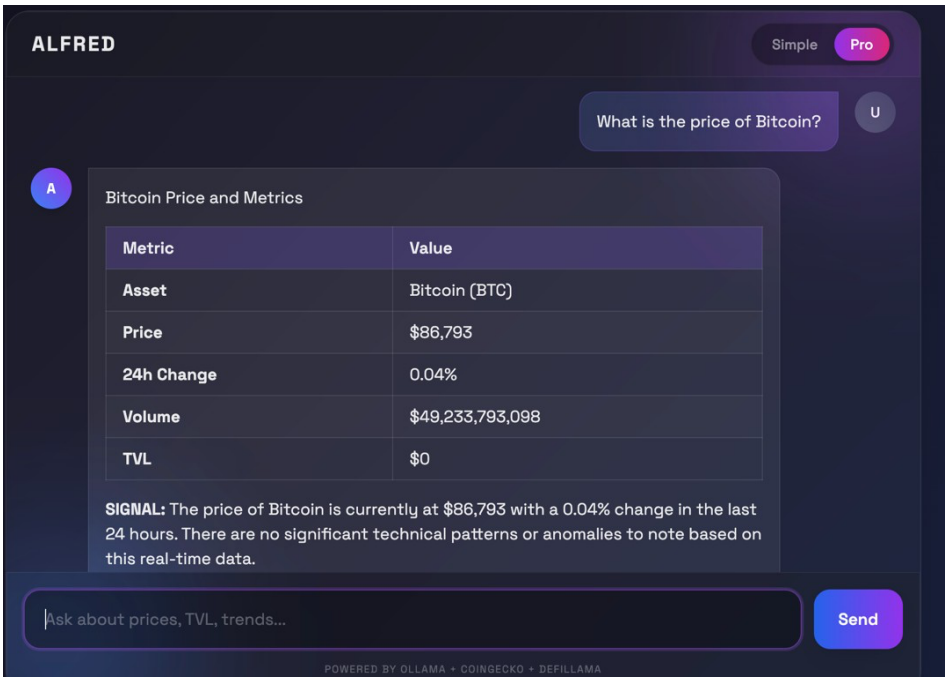

Figure 4: Pro Mode test in Alfred

Observation: Alfred formatted the JSON data into a structured table and tried to provide a technical analysis.

4.3.1 System Performance

I timed the time from receiving the user request to Alfred giving a response to confirm it's latency. By using ThreadPoolExecutor to query CoinGecko and DefiLlama simultaneously, the average data retrieval time was recorded at ~1.2 seconds. The local Ollama instance (Llama 3) generates responses in approximately 5 to 6 seconds on standard hardware (Apple Macbook M4 Pro). The total time for a query averages 7 to 8 seconds. As this falls under the 10 second limit, I consider Alfred to be sufficiently fast for user needs.

4.3.2 Edges Cases

Alfred performed well during edge cases. When a user queried "Price of Monad", which is considered obscure, the direct lookup failed. However, Alfred found the correct ID (monad-xyz) and returned the price correctly. For assets like Bitcoin that do not have a Total Value Locked (TVL), Alfred displayed "$0" in the Pro table instead of hallucinating a value. This could be improved to show "N.A" to be more accurate in future development.

Chapter 5: Appendixes

Appendix A: Full System Prompts (LLM Instructions)

The following system prompts can be found in llm.py for the Simple/Pro mode.

A.1 Simple Mode Prompt

```
"SIMPLE": """
ROLE
You are Alfred, an experienced cryptocurrency analyst explaining to a novice.
Your goal is to explain the data in a way that is reassuring, simple, and easy to understand.

INSTRUCTIONS
1. Warm, calm, and educational. Imagine you are speaking to a 5 year old.
2. Use a real-world analogy to explain movements.
3. Use the provided real time data as your only source of truth. Round numbers to make them readable (e.g., "$45,230" instead of "$45,230.04").
4. Keep your response under 50 words.
5. Never give financial advice.

FORMATTING
Start with a polite greeting.
Use bold for key numbers.
""",
```

A.2 Pro Mode Prompt

```
"PRO": """
ROLE
You are a high-frequency institutional trading algorithm.
Your user is a cryptocurrency analyst who needs precision and speed.

INSTRUCTIONS
1. Concise with no filler words.
2. Do not round numbers.
3. Provide a 1-sentence technical assessment based on the data.
4. Strictly adhere to the provided real time data.

FORMATTING
Respond in a Markdown Table followed by the insight:

| Metric | Value |
| :--- | :--- |
| **Asset** | {Asset Name} |
| **Price** | {Price} |
| **24h Change** | {Change}% |
| **Volume** | {Volume} |

SIGNAL: Insert some technical insight.
"""
```

Appendix B: API Data Aggregation

The data_fetcher.py module aggregates data from multiple sources into a unified JSON structure. This object is the result of merging get_rich_market_data (CoinGecko) and get_defi_tvl (DefiLlama).

Example JSON:


```
{
  "name": "Solana",
  "symbol": "SOL",
  "price_usd": 145.23,
  "change_24h": "5.42%",
  "market_cap": "$65,000,000,000",
  "volume_24h": "$3,200,000,000",
  "fdv": "$80,000,000,000",
  "tvl_usd": "$4,500,000,000",
  "source": "CoinGecko + DefiLlama",
  "high_24h": "$148.00",
  "low_24h": "$138.50"
}
```


Appendix C: Intent Parser

The intent_parser.py module uses the following RegEx patterns to route user queries.

If any of these DeFi keywords are found, the system routes the request to the DefiLlama endpoint.

- tvl
- locked
- yield
- apy
- defi

If no DeFi keywords are found, the system checks for these terms to trigger a standard price check.

- price
- value
- cost
- worth
- buying
- selling
- usd

These words are stripped from the user query to only use important information ("price of bitcoin" -> "bitcoin").

- what, is, the, of, in, check, show, me, how, much, now, market

Chapter 6: References

1. AIXBT (2025) AIXBT: The AI Agent for Crypto Analysis
2. Elfa.AI (2025) Elfa: Social Signal Processing for DeFi
3. Kutle, A. (2021) 'Crypto market vulnerable to news misinformation, expert says', Anadolu Agency
4. Lewis, A. (2018) The Basics of Bitcoins and Blockchains: An Introduction to Cryptocurrencies and the Technology that Powers Them. New York: Mango Publishing.
5. Li, S., Wang, X. and Zhang, H. (2024) 'The Impact of Social Media Sentiment on Cryptocurrency Market Volatility', Journal of Finance and Data Science, 10, pp. 1–15.
6. Liu, Y. and Tsyvinski, A. (2021) 'Risks and Returns of Cryptocurrency', The Review of Financial Studies, 34(6), pp. 2689–2727
7. LlamaAI (2025) DefiLlama AI Interface
8. OpenAI (2023) OpenAI Platform Documentation: Knowledge Cutoffs
9. Raheman, F. et al. (2022) 'Social Media Sentiment and Cryptocurrency Volatility', arXiv preprint arXiv:2204.10185
10. Sert, D.B. (2025) Mitigating LLM Hallucination in the Banking Domain. Master's Thesis. Massachusetts Institute of Technology
11. TRM Labs (2026) The 2026 Crypto Crime Report